Secure Session Management Documentation

Smishing Backend Security Improvement

1. Introduction

As part of my backend security contribution to the Smishing project, I worked on implementing **Secure Session Management (SSM)**. The main goal of this work was to improve how the backend handles user login sessions, token expiry, token refresh, logout, and logout from all devices.

Session management is a very important part of any web application because it controls how users stay logged in and how their access is protected. If session handling is weak, attackers may be able to reuse stolen tokens, keep access after a password change, or continue using an old session even after the user logs out.

To reduce these risks, I implemented a more secure token-based session system using:

- Access tokens
- Refresh tokens
- Token refresh endpoint
- Logout endpoint
- Logout from all devices
- Token version invalidation approach

This approach is commonly used in modern authentication systems. Large platforms and identity providers such as **Google OAuth**, **Amazon Cognito**, and **Auth0/Okta** use access tokens and refresh tokens to manage secure user sessions. Google's OAuth documentation explains that applications use access tokens to access APIs, and refresh tokens to get new access tokens after expiry. (Google for Developers) Amazon Cognito also uses refresh tokens to retrieve new access and ID tokens, and supports revocation of token-based access. (AWS Documentation)

2. Problem Before Implementation

Before this Secure Session Management work, the backend had limited control over active user sessions. Once a JWT token was issued, it could continue working until expiry. This created a security risk because if a token was stolen or exposed, the system had limited ability to immediately invalidate it.

Another problem was that the application needed better control for different logout cases. A user may want to log out from the current device only, or they may want to log out from all devices if they think their account is compromised.

The system also needed better handling for token renewal. Without refresh tokens, users may need to log in again frequently, or access tokens may be kept valid for too long, which increases security risk.

OWASP explains that JWTs are often used as authentication or session tokens, but poor implementation can lead to serious security issues, including full account compromise. (OWASP Foundation) Because of this, the backend needed stronger session control.

3. What I Implemented

I implemented Secure Session Management in the backend authentication flow. The main functionality includes:

Login with access token and refresh token

When the user logs in successfully, the backend now returns both an accessToken and a refreshToken.

Example response:

```
{
  "success": true,
  "message": "Login successful.",
  "accessToken": "jwt_access_token_here",
  "refreshToken": "jwt_refresh_token_here"
}
```

The **access token** is used for normal protected API requests. It should be short-lived because it directly gives access to protected backend routes.

The **refresh token** is used to request a new access token when the access token expires. This means the user does not need to log in again every time the short-lived access token expires.

This follows the same general idea used by OAuth-based systems, where the access token is used to access protected resources and the refresh token is used to obtain a new access token after expiry. (Google for Developers)

Refresh token endpoint

I added and tested a refresh token endpoint:

POST /api/auth/refresh-token

Example request body:

```
{
  "refreshToken": "valid_refresh_token_here"
}
```

Example response:

```
{
  "success": true,
  "accessToken": "new_access_token_here"
}
```

This improves both security and user experience. The access token can remain short-lived, while the refresh token allows the user session to continue safely.

Logout from current session

I added logout functionality for the current session:

POST /api/auth/logout

Example request body:

```
{
  "refreshToken": "refresh_token_here"
}
```

Example response:

```
{
  "success": true,
  "message": "Logged out successfully. Please remove token from client side."
}
```

This means the user can log out from the current session and the client side should remove the stored token.

Logout from all devices

I also implemented logout from all devices:

POST /api/auth/logout-all-devices

This endpoint uses the access token in the Authorization header:

Authorization: Bearer access_token_here

Example response:

```
{
  "success": true,
  "message": "Logged out from all devices successfully."
}
```

This is useful when a user changes their password, loses a device, or suspects that their account has been compromised.

4. Token Version Approach

A key part of my implementation was the **tokenVersion** approach.

In this approach, every user has a tokenVersion value stored in the database. When a token is created, the current token version is added into the token payload. Later, when the token is used, the backend compares the token version inside the JWT with the latest token version stored in the database.

If both values match, the token is accepted.

If they do not match, the token is rejected.

When the user logs out from all devices, the backend increments the tokenVersion. This makes all older tokens invalid immediately because their token version no longer matches the database value.

Example idea:

```
// Token payload example
{
  userId: "12345",
  tokenVersion: 2
```

}

If the database now has:

tokenVersion: 3

Then the old token becomes invalid.

This gives the backend better control over JWT invalidation. This is important because JWTs are normally stateless, meaning the server does not automatically remember every issued token. OWASP highlights that JWT implementations need careful handling because token misuse or weak validation can create serious security problems. (OWASP Cheat Sheet Series)

5. Real-World Examples

This type of session management is not only theoretical. It is used by many real-world systems.

Google OAuth uses access tokens and refresh tokens. The access token is used to call Google APIs, and after it expires, the refresh token is used to obtain a new access token. (Google for Developers)

Amazon Cognito uses refresh tokens to retrieve new access tokens and ID tokens. Cognito also supports token revocation, which is useful for controlling user access after logout or security changes. (AWS Documentation)

Auth0/Okta provides authentication and authorization services that support secure token-based authentication at large scale. Auth0 states that its platform handles billions of attacks and authentications, showing that token-based identity systems are widely used in production-level applications. (Auth0)

These examples show that using access tokens, refresh tokens, and revocation-style session control is a common and trusted approach in modern backend security.

6. Benefits of This Implementation

Improved security

The biggest benefit is stronger protection against stolen or old tokens. If a user logs out from all devices, old tokens become invalid. This reduces the risk of unauthorized access.

Better user experience

Users do not need to log in again every time the access token expires. The refresh token can be used to generate a new access token.

Short-lived access tokens

Access tokens can be kept short-lived. This is safer because even if an access token is exposed, it will only work for a limited time.

Better session control

The backend can now handle different session actions, such as normal logout and logout from all devices.

Better response to account compromise

If a user suspects that their account is compromised, logout from all devices can immediately invalidate old sessions.

More professional authentication flow

This implementation brings the project closer to real-world backend security practices used by modern applications and identity platforms.

7. Pros and Cons

Area	Pros	Cons / Limitations
Access token	Fast and useful for protected routes	If stolen, it can be used until expiry
Refresh token	Allows session continuation without repeated login	Must be stored securely
Logout	Allows current session to end	Client must remove token properly
Logout all devices	Invalidates old sessions using tokenVersion	Requires database check/version validation
tokenVersion	Simple way to invalidate old JWTs	It invalidates all sessions, not selected individual devices
Short-lived access token	Reduces damage from exposed token	Requires refresh flow to be working properly

8. Security Impact

This work improves the security of the Smishing backend in several important ways.

First, it reduces the risk of long-lived token misuse. Instead of relying only on one token, the system now separates short-term access and longer session continuation.

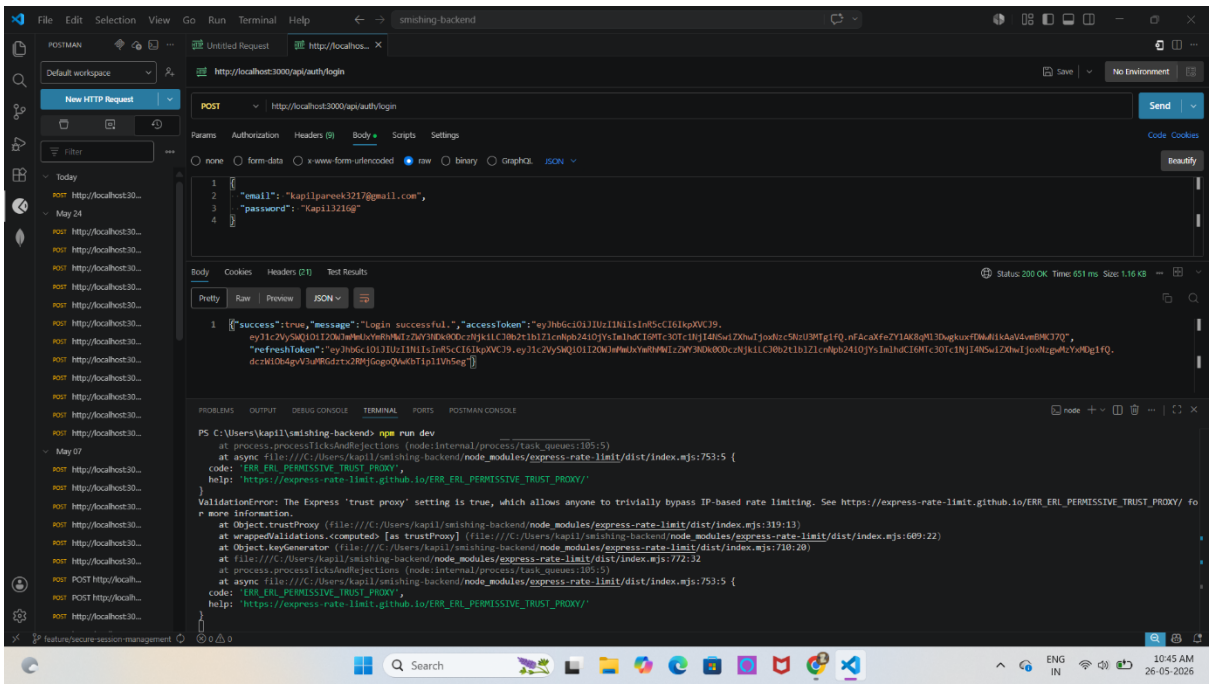
Second, it gives the backend control over old sessions. If the user logs out from all devices, the token version changes and old tokens stop working.

Third, it improves protection after password changes or suspicious account activity. In a real-world system, if a user changes their password, old sessions should not continue forever. This implementation supports that type of security control.

Fourth, it follows the same direction as widely accepted security practices. The OAuth 2.0 Security Best Current Practice document discusses updated security guidance for OAuth systems and includes stronger expectations around token handling, including refresh token protection. (IETF Datatracker) OWASP also recommends careful testing and validation of JWT-based authentication because mistakes in JWT handling can create serious vulnerabilities. (OWASP Foundation)

9. Testing Evidence from My Screenshots

I tested the implementation using Postman inside VS Code.



Test 1: Login API

Endpoint:

POST /api/auth/login

Result:

{

"success": true,

"message": "Login successful.",

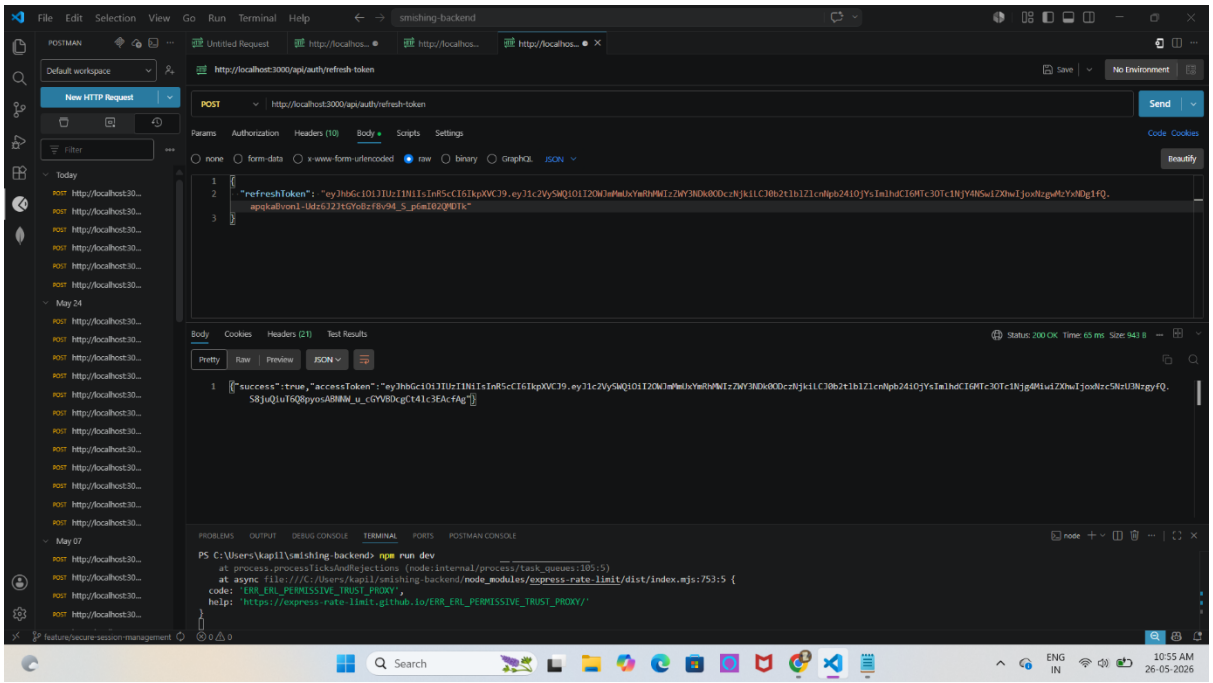
"accessToken": "...",

"refreshToken": "..."

}

This confirmed that the login API returns both access and refresh tokens.

Test 2: Refresh Token API



Endpoint:

POST /api/auth/refresh-token

Result:

{

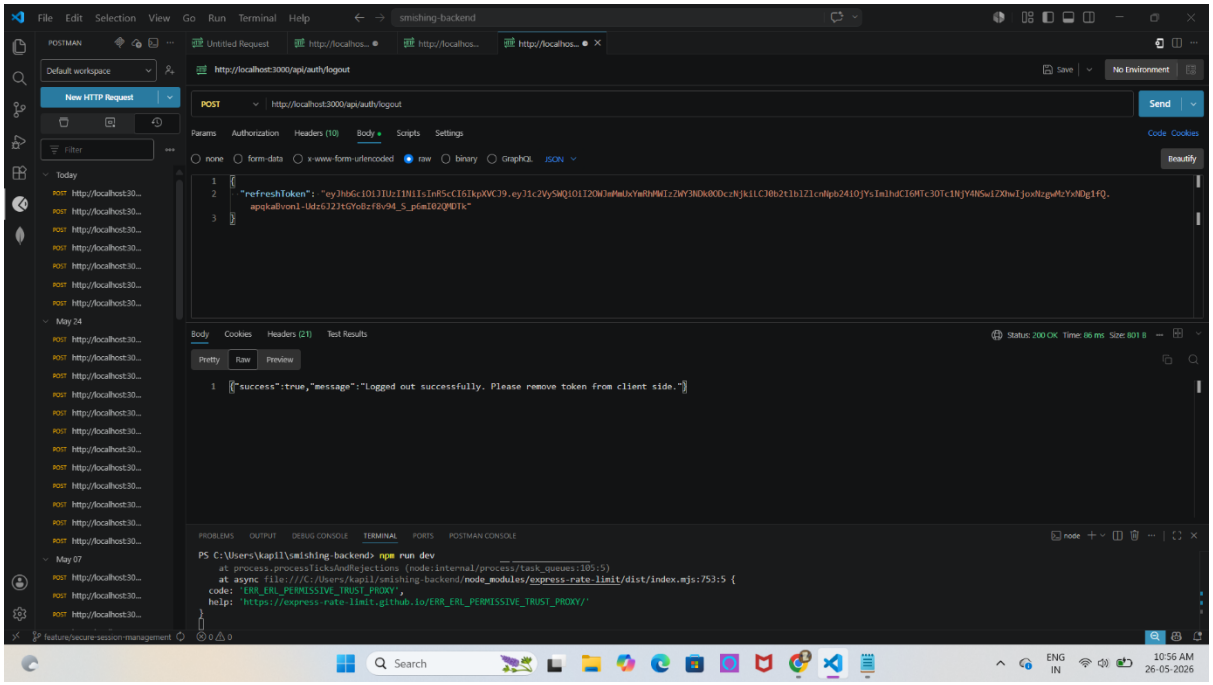
"success": true,

"accessToken": "..."

}

This confirmed that a valid refresh token can generate a new access token.

Test 3: Logout API



Endpoint:

POST /api/auth/logout

Result:

{

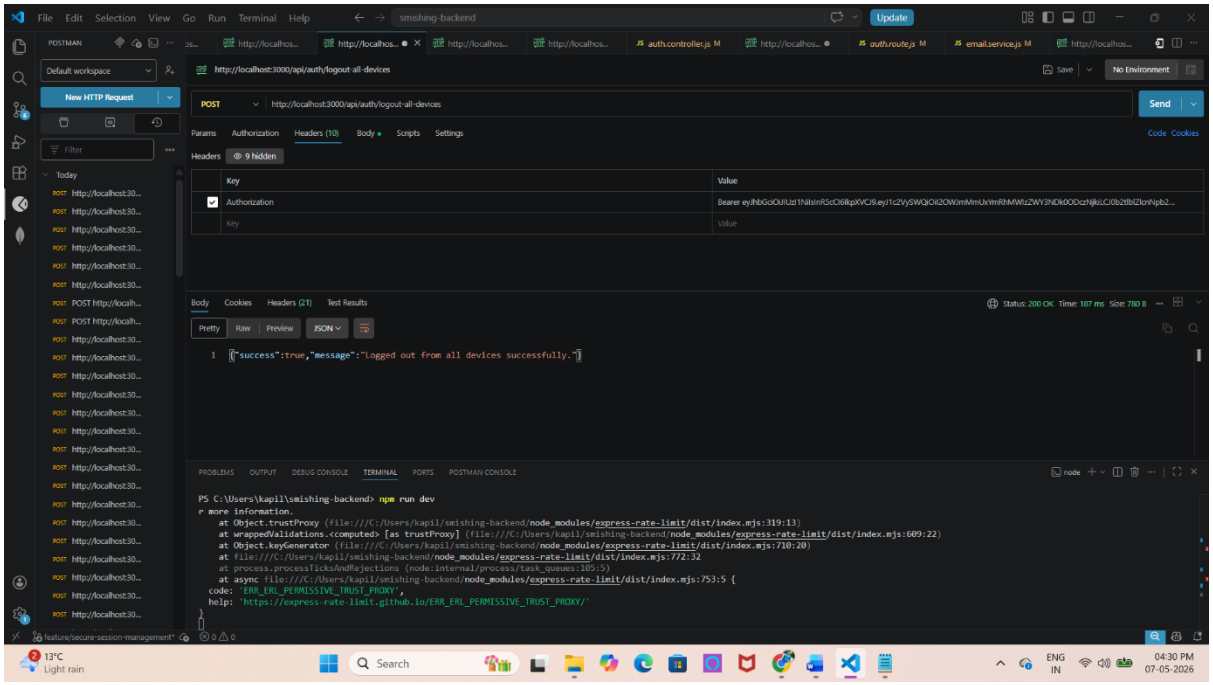
"success": true,

"message": "Logged out successfully. Please remove token from client side."

}

This confirmed that current session logout works correctly.

Test 4: Logout All Devices API



Endpoint:

POST /api/auth/logout-all-devices

Result:

{

"success": true,

"message": "Logged out from all devices successfully."

}

This confirmed that logout from all devices works and session invalidation is successful.

10. Limitations and Future Improvements

Although the current implementation improves security, there are still some possible future improvements.

One improvement is **refresh token rotation**. In refresh token rotation, every time a refresh token is used, the backend issues a new refresh token and invalidates the old one. This reduces the risk if a refresh token is stolen. OAuth security best current practice recommends stronger refresh token protection, including sender-constrained refresh tokens or refresh token rotation for public clients. (IETF Datatracker)

Another improvement is storing refresh tokens in a database with device information. This would allow logout from one specific device instead of only logout from all devices.

A third improvement is adding token reuse detection. If an old refresh token is used after rotation, the backend could detect suspicious activity and force logout from all devices.

Other future improvements include:

- Store refresh tokens in secure HTTP-only cookies.
- Add device/session tracking.
- Add refresh token expiry and cleanup.
- Add audit logs for login, logout, and token refresh.
- Add rate limiting for login and refresh endpoints.
- Fix the current trust proxy warning shown in the terminal before production deployment.
- Add automated tests for token expiry, refresh, logout, and invalid token cases.

11. Final Outcome

Overall, my Secure Session Management work made the Smishing backend more secure, reliable, and closer to real-world authentication practices.

The backend now supports login with access and refresh tokens, refreshing access tokens, logout from the current session, and logout from all devices. The tokenVersion approach gives the backend a practical way to invalidate old JWTs when needed.

This work improves account protection, reduces the risk of token misuse, and gives better control over user sessions. It also shows a strong technical contribution because authentication and session management are core security areas in any backend system.

References

1. OWASP — JSON Web Token Security Cheat Sheet. (OWASP Cheat Sheet Series)
2. OWASP — Testing JSON Web Tokens. (OWASP Foundation)
3. IETF — RFC 9700: Best Current Practice for OAuth 2.0 Security. (IETF Datatracker)
4. Google Developers — Using OAuth 2.0 to Access Google APIs. (Google for Developers)
5. Amazon Cognito — Refresh Tokens. (AWS Documentation)
6. Amazon Cognito — JWTs and Token Revocation. (AWS Documentation)

7. Auth0/Okta — Authentication and Authorization Platform. (Auth0)